MIRAI BANK PROJECT

Milestone 2 [A1]

NFT Functional Design Document

Table of Contents

0. Scope of Implementation in the Milestone 3 Prototype 3
   0.1 Response to the Milestone 1 Review Comments 5
   0.2 Comparison of Prototype and Production Implementations 6
1. Purpose of This Document 9
   1.1 Design Premises 10
2. Overview of NFT Issuance 12
   2.1 Role and Types of NFTs 12
   2.2 Design of the Minting Policy 13
   2.3 Asset Naming Rules 14
   2.4 Placement of Information (On-Chain and Off-Chain) 14
3. Design Table of Information Stored as NFTs (Corresponding to: A1) 16
   3.1 Definition of Data Types (A1-1) 16
   3.2 Definition of Data Attributes and Content (A1-2) 17
   3.3 Links, Hash Values, and Encryption Information for Stored Data (A1-3) 20
   3.4 Decryption Conditions for Displayed Data (A1-4) 22
   3.5 Time Conditions for Displaying Data (A1-5) 23
   3.6 Design Table (Consolidated) 24
4. Datum Design and Where Fields Are Held in the Prototype 27
   4.1 Datum Fields (Production) 27
   4.2 Redeemers and Decision Logic (Production) 28
   4.3 Where Fields Are Held in the Prototype 29
   4.4 Trust Assumptions 32
5. Structure and Example of CIP-25 Metadata 35
   5.1 Structure of the Manifest JSON (Production) 36
6. Detailed Design of the Minting Flow 38
   6.1 Composition of the Minting Transaction 38
   6.2 Split Minting (Production) 39
   6.3 State Updates After Minting 40
7. Basis for Estimating Network Fees 41
8. Key Design Decisions and Their Rationale 43
9. Summary 45

0. Scope of Implementation in the Milestone 3 Prototype

This document distinguishes two stages of implementation using the terms "prototype" and "production." The prototype refers to the system developed in Milestone 3. It runs on the Cardano Preprod testnet, and the opening decision is made by the server (Node.js) without using Aiken. Production refers to the production environment built in the full development that follows the prototype. It runs on Cardano Mainnet and introduces Aiken validators. Throughout this document, wherever the implementation differs between the two, the text states "prototype" or "production" explicitly, and Section 0.2 summarizes the overall comparison.

For the prototype, the scope of the user interface has been narrowed to a PC browser, individual (single-user) capsules only, one piece per capsule (a text, an image, or a PDF; images and PDFs up to 1 MB), and opening mode ① only. This chapter clarifies which parts of the NFT design defined in this document (A1) are actually implemented in the Milestone 3 prototype.

An important premise is that this chapter does not change the data design itself. The data types, attributes, Datum, and CIP-25 structure defined in Chapters 3 through 7 are consistent with the basic design established in Milestone 1 (the four-layer model, the three roles, and the three opening modes), and their correctness does not depend on the scope of the prototype. The prototype uses only some of the values and branches of the design; the rest is retained in this document so that it can be used as is when the scope is expanded.

Design Item	Treatment in the Milestone 3 Prototype	Section
Master certificate (Master NFT)	Not issued. Individual capsules correspond to mode ① and are out of scope by design (as described in Section 2.1)	2.1, Table 2-1
opening_mode	Fixed to MODE_1. The values and decision logic for MODE_2 and MODE_3 are kept in code but cannot be selected from the UI	3.2, Chapter 4
content_type	One of TEXT, IMAGE, or FILE is issued. IMAGE is limited to JPEG and PNG, and FILE to PDF; images and PDFs are limited to 1 MB and text to 100 KB (Output A2, Section 2.4)	3.2, Table 3-2
slot_no	An individual capsule always has one piece, so the value is fixed to 001	2.3, 3.2
quorum	Not used, because master certificates are not issued	3.2, Table 3-3
Split minting (Section 6.2)	Does not occur, because each capsule has one NFT	6.2
Datum (script UTxO)	Not created. Aiken is out of scope for the prototype, so the Datum fields are held in the CIP-25 metadata (authoritative) and the server database (working copy), and the server makes the decision	4.3, 4.4
Signature condition of the minting policy	A native script consisting only of the user's (capsule manager's) wallet signature and a time condition. Signing with a service-side minting key is introduced in production	2.2
Manifest JSON	Not created. Because each capsule has exactly one piece, the ciphertext can be reached directly from content_uri in the NFT metadata. manifest_uri and manifest_hash are not set	2.4, 5.1

Design Item	Treatment in the Milestone 3 Prototype	Section
Detection and notification of openable capsules	The scheduled job (node-cron) and notification e-mail are not implemented. unlock_at is evaluated at the time of the opening operation	3.5
Display of NFT information (required in the prototype)	SCREEN-03 (Dashboard) and SCREEN-09 (Opening Viewer) show the asset name, the policy ID, and a link to a Cardano explorer. This is required to meet the Milestone 3 acceptance criteria (NFT display and a per-user NFT list) and is reflected in Output B1	Chapter 5
Minimum opening date/time (prototype test setting)	For functional testing and for recording the demonstration video, a test setting allows the minimum to be shortened to five minutes. The default (current time plus one hour) is unchanged. Reflected in Output B1	3.5

Table 0-1 Scope of Implementation in the Milestone 3 Prototype

Chapters 1 through 9 describe the full design, including the items marked above as out of scope, together with the responses to the Milestone 1 review comments (Sections 0.1, 4.3, and 4.4). When implementing Milestone 3, it is sufficient not to build the code branches and UI paths for the items marked as out of scope; no change to the data model itself is required.

0.1 Response to the Milestone 1 Review Comments

The Milestone 1 outputs were approved by the Catalyst reviewers in September 2026. At that time, the reviewers raised two points to be addressed in Milestone 2. This document responds to the first point: where the fields used for the opening decision are held during

the prototype stage, and what trust assumptions apply. Table 0-2 shows where each point is addressed. The second point (IPFS pinning operations and the Arweave migration policy) is addressed in Output A2 (Data Storage Functional Design Document).

Review Comment	Response	Reference
Where fields such as the opening mode are held in the prototype	The prototype does not use Aiken, so no on-chain Datum exists. For each Datum field, this document defines where it is held in the prototype (CIP-25 metadata and the server database), how the two share roles, and how they are reconciled before a key is released	Section 4.3, Table 4-2, Figure 4-2
Stating the trust assumptions	For both the prototype and production (after the Aiken migration), this document distinguishes, item by item, what third parties can verify independently from what relies on trust in the operator (and in external services), and describes the mitigations used in the prototype	Section 4.4, Table 4-3, Figure 4-3

Table 0-2 Milestone 1 Review Comments and Responses in This Document

0.2 Comparison of Prototype and Production Implementations

Table 0-3 summarizes the differences between the prototype and production implementations of the NFT functions defined in this document. The data design (field names, types, and meanings) is common to both. The main differences lie in which values and branches are used, who makes the opening decision, who signs the minting policy, and whether a manifest is used.

Item	Prototype (Milestone 3)	Production
Network	Cardano Preprod testnet	Cardano Mainnet
Capsules in scope	Individual capsules only. One piece per capsule: a text, an image (JPEG or PNG), or a PDF (images and PDFs up to 1 MB); opening mode ① only (Table 0-1)	Individual and group capsules. Text, images, and files (limits by type and plan); opening modes ①, ②, and ③
Certificates issued	Piece certificates only (one per capsule)	Piece certificates plus master certificates (modes ② and ③), with split minting according to the number of pieces
Minting policy	Native script with the user's wallet signature and a time condition (one policy per capsule)	Signature of a service-side minting key plus a time condition (one policy per capsule)
CIP-25 metadata	Structure in Chapter 5 (only the values used by the prototype are set; manifest_uri is not set)	Structure in Chapter 5 (all fields used)
Manifest JSON	Not created	Created for each capsule and stored in decentralized storage (Section 5.1)
Display of NFT information	Asset name, policy ID, and an explorer link on SCREEN-03 (Dashboard) and SCREEN-09 (Opening Viewer)	The same, plus details of the minting transaction and the verification status
Detection and notification of openable capsules	Evaluated at the time of the opening operation (no scheduled job or notification)	Detected by a scheduled job, followed by a notification e-mail
content_uri	ipfs:// (Blockfrost IPFS)	ar:// (the IPFS copy is recorded in the manifest's mirror_uris)

Item	Prototype (Milestone 3)	Production
Datum	Not created. The same fields are held in the CIP-25 metadata (authoritative) and the server database (working copy) and are reconciled before a key is released (Section 4.3)	Recorded in the Datum of a script UTxO (Section 4.1)
Who makes the opening decision	The server (Node.js), within the key release API. The content key (CEK) is returned over an encrypted connection (TLS)	An Aiken validator verifies the opening transaction (keys are released by the key service)
Time condition	Server time (NTP-synchronized)	Validity range of the transaction
Confirmation of NFT holding	Blockfrost query of the signer's assets	The validator verifies the NFT in the inputs of the opening transaction
Trust assumptions	The operator is trusted for both the opening decision and key release (Section 4.4)	The opening decision is verifiable by third parties; only key release relies on the operator
Chain queries (Blockfrost)	Free Starter plan by default; the Hobby plan if a second project cannot be created (Output A2, Section 2.5.3)	Selected according to production usage
Payment of fees	The capsule manager (test ADA)	The manager, or a plan in which the service pays on the user's behalf

Table 0-3 Comparison of Prototype and Production Implementations (NFT Functions)

1. Purpose of This Document

This document is Milestone 2 ("Prototype detailed design") Output A1, "Prototype functional design: NFT function design document." Building on the NFT standard selected in Milestone 1 (CIP-25, Milestone 1 Output A1) and the linkage between blockchain functions and the UI (Milestone 1 Output C1), it finalizes the design of the information and data used to issue NFTs, at a level of detail that allows implementation in Milestone 3.

The acceptance criterion is a design table of the information stored as NFTs, which must include at least the following five items. Chapter 3 organizes each of these five items in its own section, and Section 3.6 consolidates them into a single design table.

- **A1-1:**

  Definition of data types

- **A1-2:**

  Definition of data attributes and content

- **A1-3:**

  Links, hash values, and encryption information for the stored data

- **A1-4:**

  Decryption conditions for the displayed data

- **A1-5:**

  Time conditions under which the data is displayed

The scope of this document is the information stored as NFTs and how the NFTs are issued. Details of encryption, key management, and storage are covered in Output A2 (Data

Storage Functional Design Document), and on-screen display and operations in Output B1 (User Interface Functional Design Document). The three outputs reference one another and share common terms and field names.

1.1 Design Premises

Premise	Effect on This Document
NFT as a certificate; script as a vault	The NFT metadata records, as proof, what opens, when, and under which conditions. Opening is enforced by the script (in the prototype, by a server-side decision). The decryption key itself is never placed in the NFT.
Holding the NFT confers the right to open	The Datum holds nft_policy_id and nft_asset_name rather than the owner's key hash. Transferring the NFT means transferring the right to open.
Staged migration of the time lock from Node.js to Aiken	In the prototype, the server evaluates the time condition; Aiken is adopted in production. The Datum fields are defined so that they can be used as is after the Aiken migration, and in the prototype the same fields are held in the CIP-25 metadata and the server database (Section 4.3).
Storage: IPFS (prototype), Arweave (production)	Links to stored data accept both ipfs:// and ar://. In the prototype, pinning uses Blockfrost IPFS (Output A2, Section 2.2).
Client-side AES-256 encryption	The NFT records only information that is needed for decryption but is not secret, such as the encryption method, the IV, and hashes.
Three opening modes for group capsules	The opening mode and the master certificate (Master NFT) are reflected in both the NFT metadata and the Datum.

Premise	Effect on This Document
Adoption of CIP-25 (Milestone 1 Output A1)	Metadata is recorded under transaction metadata label 721. CIP-68 is not adopted.

Table 1-1 Design Premises for This Document

2. Overview of NFT Issuance

2.1 Role and Types of NFTs

The NFT in this service is a certificate that allows third parties to verify when, by whom, and under which opening conditions the content recorded in a piece was sealed. The content itself is not placed in the NFT. The encrypted content is stored in decentralized storage, and its location (URI) and fingerprint (hash value) are recorded in the NFT. Table 2-1 shows the two types of NFTs issued.

Type	Unit of Issuance	Holder	Role
Piece certificate (Piece NFT)	One per piece	The registrant of the piece (MEMBER or individual user)	Certifies the content, location, and opening conditions of the piece. Holding it is the basis of the right to open.
Master certificate (Master NFT)	One per group manager (modes ② and ③ only)	GROUP_OWNER or CAPSULE_OWNER	A certificate that serves as the key for a manager to open the entire capsule. Not issued in mode ①.

Table 2-1 Types of NFTs Issued

Internally, an individual capsule (a SOLO group) is handled by the same mechanism as a group capsule. Therefore, a piece certificate is issued for each piece even in an individual capsule. A master certificate is not issued because the manager and the registrant are the same person (the capsule is treated as mode ①).

2.2 Design of the Minting Policy

On Cardano, an NFT is uniquely identified by its policy ID and asset name. This service prepares one minting policy per capsule and issues all piece certificates and master certificates of that capsule under the same policy ID. This establishes a simple correspondence ("NFTs with the same policy ID belong to the same capsule"), which makes both the Datum-side verification (Chapter 4) and wallet display easier to handle.

Item	Design
Policy type	Native script. A native script combining a signature condition and a time condition satisfies the requirements without an Aiken minting validator.
Signature condition	Production: the signature of a service-side minting key is required. The capsule manager's wallet signature pays the fees and minimum ADA, and the service-side key guarantees the legitimacy of the issuance. Prototype: the capsule manager's wallet signature only (see below).
Time condition	Minting becomes impossible after a slot a fixed period (default: 24 hours) after the capsule is finalized (a "before" condition). After that, no additional minting or burning is possible under the same policy, so the supply is fixed.
Storing the policy ID	The generated policy script and policy ID are stored in the database (Project.policyId and Project.policyScript). The same value is written to nft_policy_id in the Datum.
Reissuance	Not possible after the time condition has passed. If minting fails, it is retried with the same policy within the time condition (Chapter 6).

Table 2-2 Design of the Minting Policy

In the prototype, the signature condition of the policy is simplified to the capsule manager's wallet signature plus the time condition, and no service-side minting key is used. This removes the need for server-side secret key management and for assembling a

two-party signature combining the wallet signature and a service-side signature. It also matches the Milestone 3 acceptance criterion of user-based NFT issuance.

In this configuration, anyone can create an NFT with an asset name of the same format. However, the opening decision only considers the combination of policy ID and asset name recorded in the database at sealing (Section 4.3), and the CIP-25 metadata values are further reconciled with the database values. A self-made NFT therefore cannot cause a key to be released. Assurance to third parties that a certificate was issued by the service is provided in production by introducing the service-side minting key.

2.3 Asset Naming Rules

An asset name is at most 32 bytes. It is a human-readable identifier, with the capsule's short ID at the beginning so that the NFTs of the same capsule appear together in a wallet.

```
Piece certificate  : MB-{capsuleShortId}-P{slotNo}    e.g. MB-7K3QX2-P007
Master certificate : MB-{capsuleShortId}-M{managerNo}  e.g. MB-7K3QX2-M01

capsuleShortId : 6 alphanumeric characters derived from Project.id (cuid); regenerated on collision
slotNo       : Sequence number of the PieceSlot (3 digits, zero-padded)
managerNo    : Sequence number of the master certificate (2 digits, zero-padded)
Characters   : Uppercase letters, digits, and hyphens only (ASCII, max. 32 bytes)
```

Figure 2-1 Asset Naming Rules

2.4 Placement of Information (On-Chain and Off-Chain)

Information stored as NFTs is distributed across four places: transaction metadata (CIP-25), the Datum of a script UTxO, a manifest in decentralized storage, and the server database. The Datum is created only in production (after the Aiken migration); in the

prototype, the information is placed in the remaining three places (Section 4.3). Figure 2-2 shows the overall picture.

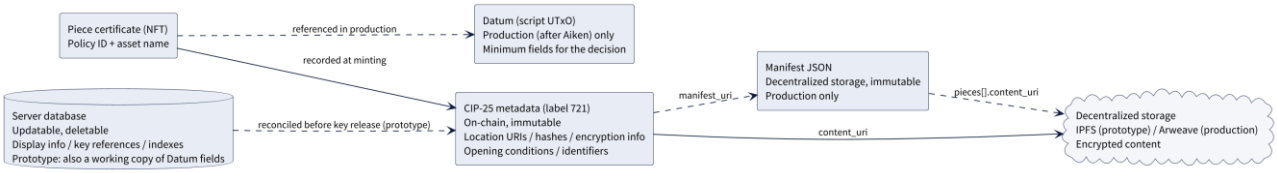


Figure 2-2 Placement of Information Around the NFT

Location	Characteristics	Information Placed
CIP-25 metadata	On-chain, immutable, readable by anyone	Information to be kept as proof: location URIs, hashes, encryption method, time conditions, opening mode, and identifiers.
Datum (script UTxO)	On-chain, for machine decisions (production only)	The minimum fields needed for the opening decision (Chapter 4). After the Aiken migration, the validator can decide using only these. Not created in the prototype; the same fields are held in the database.
Manifest JSON	Decentralized storage, immutable	The list of all pieces in the capsule and the hash of each piece. One per capsule, referenced from the NFT via manifest_uri (production only).
Server database	Off-chain, updatable, deletable	Supplementary display information (display names, progress, notification addresses), references to wrapped keys, and search indexes. In the prototype, it also holds a working copy of the fields used for the opening decision (Section 4.3). Deletion requests, such as those under the GDPR, are handled here.

Table 2-3 Locations and Characteristics of Information

3. Design Table of Information Stored as NFTs (Corresponding to: A1)

This chapter defines, section by section, the five items required by the acceptance criterion, and Section 3.6 consolidates them into a design table. As a rule, the same field name (key) is used in the CIP-25 metadata, the Datum, the database schema, and API responses, written in snake_case.

3.1 Definition of Data Types (A1-1)

The types available in Cardano transaction metadata are limited; in particular, each string element is limited to 64 bytes. This service uses the types in Table 3-1, and strings longer than 64 bytes (such as IPFS CIDs) are split into an array of strings (a notation permitted by CIP-25).

Type (This Document)	Representation in Metadata	Constraint	Example Use
string	JSON string	Up to 64 bytes in UTF-8	name, mediaType, capsule_type
long_string	Array of strings	Each element up to 64 bytes; concatenated to form one value	content_uri, manifest_uri, description
hex	Hexadecimal string (lowercase)	Even number of digits; split like long_string if over 64 bytes	content_hash, iv, policy_id
int	JSON integer	64-bit signed; no decimals or exponents	unlock_at, version, quorum

Type (This Document)	Representation in Metadata	Constraint	Example Use
enum	JSON string	Only the values listed in this document	opening_mode, enc.alg, capsule_type
uri	long_string	Scheme is ipfs:// or ar:// only	content_uri, image, manifest_uri
object	JSON object	Keys in lowercase letters and underscores	enc, files[]
array	JSON array	Element type fixed for each field	files, managers

Table 3-1 Definition of Data Types

On the Datum side (Plutus Data), the same fields are expressed as Int, ByteArray, and List. The correspondence is shown in Section 4.1. All dates and times are held as UTC POSIX timestamps in milliseconds; strings with time zones are not recorded on-chain.

3.2 Definition of Data Attributes and Content (A1-2)

Table 3-2 defines the attributes included in the CIP-25 metadata of a piece certificate. In addition to the fields required by CIP-25 (name and image), service-specific attributes are added. Personally identifiable information (names, e-mail addresses, and full wallet addresses) is never placed on-chain.

Key	Type	Required	Content
name	string	Yes	Display name in the form "{capsule name} #007". Required by CIP-25.

Key	Type	Required	Content
image	uri	Yes	Thumbnail image. The service's common capsule image (fixed IPFS CID) is the default; no image that suggests the content is used. Required by CIP-25.
mediaType	string	Yes	MIME type of image. Default: image/png
description	long_string	—	Description of the capsule (entered by the manager, within what may be made public). Up to 256 characters.
version	int	Yes	Version of this metadata design. Initial version: 1
capsule_id	string	Yes	Project.id (cuid)
capsule_type	enum	Yes	SOLO or GROUP
piece_id	string	Yes	Piece.id (cuid)
slot_no	int	Yes	Sequence number of the PieceSlot (matches the asset name)
kind	enum	Yes	PIECE or MASTER (master certificate)
content_type	enum	Yes	TEXT, IMAGE, or FILE. Determines how the content is displayed after decryption.
content_uri	uri	Yes	Location of the encrypted content (3.3)
content_hash	hex	Yes	SHA-256 of the ciphertext (3.3)
plain_hash	hex	Yes	SHA-256 of the plaintext. Detects tampering and wrong keys after decryption (3.3)
enc	object	Yes	Encryption information (3.3)
manifest_uri	uri	Yes (production)	Location of the manifest JSON for the whole capsule. Not set in the prototype

Key	Type	Required	Content
unlock_at	int	Yes	Date/time from which opening is possible (UTC, milliseconds) (3.5)
opening_mode	enum	Yes	MODE_1, MODE_2, or MODE_3 (3.4, 3.5)
sealed_at	int	Yes	Date/time the capsule was sealed (UTC, milliseconds). Assigned by the server.
registrant	string	Yes	The first 8 and last 6 characters of the registrant's wallet address (e.g. addr_tes…x7q2kd). The full address is not recorded.
files	array	—	The CIP-25 files array. One entry for the encrypted content: { name, mediaType: application/octet-stream, src }.

Table 3-2 Data Attributes of the Piece Certificate

A master certificate (kind = MASTER) does not have content_uri, content_hash, plain_hash, enc, piece_id, slot_no, or content_type; instead, it has the attributes in Table 3-3.

Key	Type	Content
manager_no	int	Sequence number of the master certificate (matches the asset name)
manager_count	int	Number of managers N at the time of issuance
quorum	int	Number of agreeing managers required to execute mode ③ (the default defined in Milestone 1 Output C1, or a value specified for the capsule). 0 in mode ②
covers	array	List of asset names of the piece certificates that this certificate can open together

Table 3-3 Attributes Specific to the Master Certificate

3.3 Links, Hash Values, and Encryption Information for Stored Data (A1-3)

This section defines the information for reaching the stored data from the NFT and for confirming that the retrieved data has not been tampered with. The decryption key itself is never recorded on-chain. Only information that someone who has the key needs in order to decrypt correctly, and that is safe to make public, is recorded.

3.3.1 Links (Location)

Key	Format	Description
content_uri	ipfs://{CID} or ar://{TxID}	The encrypted content. IPFS in the prototype; Arweave as the primary store in production, with mirrors listed in the manifest (Output A2, Chapter 4).
manifest_uri	ipfs://{CID} or ar://{TxID}	The manifest JSON for the whole capsule, listing the content_uri, hashes, and mirror URIs of all pieces (production only).
image	ipfs://{CID}	Thumbnail. A fixed CID common to the service.

Table 3-4 Link Information

In production, IPFS CIDs are recorded as CIDv1 (base32); in the prototype, the CID returned by Blockfrost IPFS is recorded as is. The base32 form of a CIDv1 is about 59 characters and fits within 64 bytes, but in preparation for future changes to the hash function, all URI fields are handled as long_string.

3.3.2 Hash Values (Integrity)

Key	Target	When Verified and Why
content_hash	SHA-256 of the ciphertext (the	Immediately after retrieval, to confirm that

Key	Target	When Verified and Why
	uploaded byte sequence itself)	the data retrieved from storage is identical to that at sealing. It can be verified without the decryption key, so it also serves as proof of existence for third parties.
plain_hash	SHA-256 of the plaintext (the byte sequence before encryption)	Immediately after decryption, to detect wrong keys, decryption errors, and tampering at the plaintext stage. It is meaningless to a third party who does not know the plaintext and does not help guess the content.
manifest_hash	SHA-256 of the manifest JSON	Immediately after retrieving the manifest, to detect replacement of the piece list. Recorded independently of the enc object of the piece certificates (production only).

Table 3-5 Hash Information

3.3.3 Encryption Information (enc Object)

Key	Type	Value and Description
enc.alg	enum	AES-256-GCM (fixed). As authenticated encryption, the decryption process itself also detects tampering.
enc.iv	hex	Initialization vector (96 bits = 12 bytes), randomly generated for each piece. Safe to make public.
enc.tag_len	int	Length of the authentication tag (bits). Default: 128
enc.key_ref	string	Reference ID of the wrapped key in the key service, not the key itself. Format: k1:{capsule_id}:{piece_id}
enc.kdf	enum	NONE (default). Keys are randomly generated and stored wrapped, so no key derivation function is used. The field is kept to distinguish the earlier password-based derivation approach.

Key	Type	Value and Description
enc.wrap	enum	Key wrapping method: AES-KW-256 in the prototype, KMS in production (Output A2, Chapter 3)

Table 3-6 Encryption Information (enc)

3.4 Decryption Conditions for Displayed Data (A1-4)

A decryption condition defines who can receive the release of the decryption key (CEK) and in which state. Because the key is not in the NFT, a person who meets the conditions receives the key from the server (in production, a key service backed by the Aiken validator) and decrypts the content on their own device. The conditions are determined by the opening mode.

Condition	Who	Holds	Additional Condition	What Can Be Decrypted
C-1	The registrant of the piece	The piece certificate of that piece	Current time ≥ unlock_at	Their own piece only
C-2	Group manager (mode ②)	A master certificate	Current time ≥ unlock_at	All pieces in the capsule
C-3	Group manager (mode ③)	A master certificate	Number of agreeing managers ≥ quorum (no time condition)	All pieces in the capsule
C-4	A delegated viewer	Registered in ViewerPermission (WALLET type)	The capsule is OPENED	The delegated pieces (viewing after opening only)

Table 3-7 Decryption Conditions

In mode ①, only C-1 applies. In mode ②, C-1 and C-2 apply; in mode ③, C-1, C-2, and C-3

apply. C-4 is not a decryption condition but a condition that permits viewing of decrypted content after opening, and it does not involve releasing a key. The flow of the decision is the same as the method defined in Milestone 1 Output C1, Chapter 5.

In the prototype, holding is determined by the server querying the assets held by the signer's wallet through Blockfrost. After the Aiken migration, the validator verifies that the relevant NFT is included in the inputs of the opening transaction. If an NFT is transferred to a third party, the recipient becomes the new holder and satisfies C-1 or C-2.

3.5 Time Conditions for Displaying Data (A1-5)

Key	Type	Set By and When	Meaning
unlock_at	int	Set by the capsule manager on SCREEN-04 (Capsule Creation); cannot be changed after sealing	The date/time from which opening is possible. Before this, no decryption is performed except under C-3. Recorded in UTC milliseconds and converted to the device's time zone (default JST) for display.
opening_mode	enum	Same as above	MODE_1, MODE_2, or MODE_3. Indicates whether an exception to the time condition (mode ③) exists.
sealed_at	int	Assigned by the server when sealing is confirmed	The sealing date/time; proof of when the capsule was sealed. unlock_at − sealed_at is the storage period.
grace_until	int	Calculated automatically by the server (optional)	unlock_at plus a grace period. Not used in the prototype (0). Reserved

Key	Type	Set By and When	Meaning
			for purposes such as the deadline for resending opening notifications.

Table 3-8 Time Conditions

The time condition is evaluated against server time (NTP-synchronized). In production, an opening monitoring job (node-cron, every hour on the hour) detects capsules whose unlock_at has passed, updates their state to OPENABLE, and sends notifications. In the prototype, no scheduled job or notification is used, and unlock_at is evaluated at the time of the opening operation. After the Aiken migration, the validator verifies that the lower bound of the transaction's validity range is at or after unlock_at. The minimum unlock_at is the sealing time plus one hour and the maximum is the sealing time plus 100 years; values outside this range are rejected as input errors on SCREEN-04 (Capsule Creation). In the prototype, a test setting allows the minimum to be shortened to five minutes for functional testing and the demonstration video.

3.6 Design Table (Consolidated)

Sections 3.1 through 3.5 are consolidated into a single design table organized by the five acceptance criteria. The "Location" column shows where each field is recorded (M: CIP-25 metadata, D: Datum, F: manifest, DB: server database).

Acceptance Criterion	Fields (Keys)	Type	Location	Key Points
A1-1 Data types	string, long_string, hex, int, enum, uri,	—	M	The 64-byte limit is handled with

Acceptance Criterion	Fields (Keys)	Type	Location	Key Points
	object, array			long_string. Dates and times are int values in UTC milliseconds.
A1-2 Attributes and content	name, image, mediaType, description, version	string, etc.	M	Fields required by CIP-25. image is a common fixed image.
	capsule_id, capsule_type, piece_id, slot_no, kind, content_type	string, enum, int	M, DB	Identifiers and display method. No personal information.
	registrant, sealed_at	string, int	M, DB	Shortened address of the registrant and the sealing time.
	manager_no, manager_count, quorum, covers	int, array	M, D	Master certificates only.
A1-3 Links, hashes, encryption	content_uri, manifest_uri	uri	M, D, F, DB	ipfs:// or ar://. Recorded in the Datum as content_cid.
	content_hash, plain_hash, manifest_hash	hex	M, F, DB	Verified in two stages: immediately after retrieval and immediately after decryption.
	enc.alg, enc.iv, enc.tag_len, enc.key_ref, enc.kdf, enc.wrap	object	M, DB	The key itself is not included; key_ref references the key service.

Acceptance Criterion	Fields (Keys)	Type	Location	Key Points
A1-4 Decryption conditions	opening_mode, kind (PIECE or MASTER), quorum, managers	enum, int, array	M, D	C-1 to C-4, decided by NFT held, time, and number of agreeing managers.
A1-5 Time conditions	unlock_at, sealed_at, grace_until	int	M, D, DB	UTC milliseconds. Decided by the server in the prototype; verified through the validity range in production.

Table 3-9 Design Table of Information Stored as NFTs (Consolidated)

4. Datum Design and Where Fields Are Held in the Prototype

The Datum is the minimum data that an Aiken validator refers to in production to decide whether to permit opening. Table 4-1 shows its final form, which extends the design established in Milestone 1 with the fields required for the three opening modes.

Because Aiken is out of scope for the prototype, no script UTxO is created and no on-chain Datum exists. Sections 4.1 and 4.2 are production (post-Aiken-migration) specifications. Section 4.3 shows where the same fields are held in the prototype, and Section 4.4 shows on whose trust the decision relies.

4.1 Datum Fields (Production)

Field	Type (Plutus)	Description
unlock_time	Int	Date/time from which opening is possible (POSIX milliseconds). Same value as unlock_at in CIP-25.
content_cid	ByteArray	The identifier (CID or TxID) of content_uri without the scheme, held as bytes.
nft_policy_id	ByteArray	The minting policy ID of this capsule (28 bytes).
nft_asset_name	ByteArray	The asset name of the piece certificate for this piece.
opening_mode	Int	1, 2, or 3
master_policy_id	ByteArray	The policy ID of the master certificates. In this design, the same as nft_policy_id (same

Field	Type (Plutus)	Description
		policy). Empty in mode ①.
master_asset_prefix	ByteArray	The asset name prefix of the master certificates (MB-{capsuleShortId}-M). Anyone holding an asset matching this prefix is treated as a manager. Empty in mode ①.
managers	List<ByteArray>	List of the managers' payment key hashes (PKH), used to count agreeing managers in mode ③.
quorum	Int	Number of agreeing managers required in mode ③. 0 in modes ① and ②.
content_hash	ByteArray	SHA-256 of the ciphertext (32 bytes). Holds on-chain proof of integrity independently of the metadata.

Table 4-1 Datum Fields (Final Form)

4.2 Redeemers and Decision Logic (Production)

```
Redeemer:
  OpenAsHolder              -- C-1: open as the holder of a piece certificate
  OpenAsManager             -- C-2: open as the holder of a master certificate (time condition applies)
  EmergencyOpen { approvals: Int } -- C-3: emergency opening with the number of agreeing managers (no time condition)

Decision (all conditions must hold):
  OpenAsHolder  : inputs contain the NFT (nft_policy_id, nft_asset_name)
                  AND validity_range.lower >= unlock_time
  OpenAsManager : opening_mode in {2,3}
                  AND inputs contain an asset starting with master_asset_prefix
                  AND validity_range.lower >= unlock_time
  EmergencyOpen : opening_mode == 3
                  AND inputs contain an asset starting with master_asset_prefix
                  AND at least quorum of managers have signed the transaction
```

Figure 4-1 Redeemers and Decision Conditions (Specification for the Aiken Migration)

In the prototype (Node.js phase), the server performs the same decision within the key release API (/api/keys/:pieceId/release, Output A2, Chapter 6). The check of input NFTs is replaced by a Blockfrost query of held assets, and the count of signatures is replaced by the number of approval records collected in the mode ③ approval flow. By keeping the Datum field names and meanings unchanged, the difference at the time of the Aiken migration is limited to the implementation of the validator.

4.3 Where Fields Are Held in the Prototype

In the prototype, each Datum field is held in two places: the CIP-25 metadata and the server database. The CIP-25 metadata is recorded in the sealing (minting) transaction, so after sealing no one, including the operator, can rewrite it. The server database, on the other hand, can be updated by the operator. Based on this difference, the two share roles as follows.

- The CIP-25 metadata is authoritative:

  The authoritative values used as the basis of the opening decision, such as unlock_at, opening_mode, content_uri, and content_hash, are the values recorded on-chain at sealing.

- The server database is a working copy for the decision:

  For responsiveness, the server makes its decision using the database values, but immediately before releasing a key it reconciles them with the CIP-25 metadata values retrieved through Blockfrost. If they do not match, the key is not released, and the result is recorded in AccessLog as DENIED_MISMATCH.

- **Non-public fields are kept only in the database:**

  managers (the list of managers' payment key hashes) could identify individuals and is therefore not recorded in the CIP-25 metadata. The count of agreeing managers in mode ③ is confirmed using approval records in the database.

Table 4-2 shows where each field is held. The database columns are those of the tables defined in Section 6.3 (Table 6-2) and in Output A2, Section 3.6.

Datum Field	Where Held in the Prototype (DB)	Corresponding CIP-25 Metadata	Reconciled Before Key Release	Used in the Milestone 3 Prototype
unlock_time	Project.unlockAt	unlock_at	Yes	Yes
content_cid	Piece.contentUri (production: StorageObject.uri with isPrimary)	content_uri (compared without the scheme)	Yes	Yes
nft_policy_id	Project.policyId	Policy ID key under label 721	Yes	Yes
nft_asset_name	Piece.assetName	Asset name key under the policy ID	Yes	Yes
opening_mode	Capsule.openingMode	opening_mode	Yes	Fixed to MODE_1
master_policy_id	Project.policyId (same policy)	Policy ID key under label 721	Yes	No
master_asset_prefix	Derived from MasterToken.assetName	Asset names with kind = MASTER	Yes	No

Datum Field	Where Held in the Prototype (DB)	Corresponding CIP-25 Metadata	Reconciled Before Key Release	Used in the Milestone 3 Prototype
managers	GroupMembership (role) and MasterToken.managerWallet	Not recorded (protection of personal information)	DB only	No
quorum	Capsule.quorum	quorum (master certificate)	Yes	No
content_hash	Piece.contentHash	content_hash	Yes	Yes

Table 4-2 Where Datum-Equivalent Fields Are Held in the Prototype

Figure 4-2 shows the order of checks for key release in the prototype. Placing the reconciliation (step 5) immediately before key release prevents a key from being released under conditions different from those at sealing, even if database values are changed by mistake or on purpose.

```
POST /api/keys/:pieceId/release (order of checks in the prototype)
  1. Signature    Determine the signer's address from the CIP-30 signData signature
  2. Read         Read Project.unlockAt, Capsule.openingMode, Piece.assetName, etc. from the DB
  3. Time         Server time (NTP-synchronized) >= unlockAt
  4. Holding      Blockfrost: the signer's assets include (policyId, assetName)
  5. Reconcile    Blockfrost: fetch the CIP-25 metadata of the asset and confirm that
                  unlock_at, opening_mode, content_uri, and content_hash match the DB
  6. Release      Only if 1-5 all hold, return the CEK over an encrypted connection (TLS)
                  (in production, re-wrapped with the device's ephemeral public key)
  7. Log          Record the result (GRANTED, DENIED_TIME, DENIED_HOLDER, DENIED_MISMATCH) in
AccessLog
```

Figure 4-2 Order of Checks for Key Release in the Prototype

When migrating to Aiken in production, the values in Table 4-2 are also written to the

Datum of a script UTxO in the sealing transaction, and the checks in steps 3 and 4 move to the validator. Because the field names and meanings are identical to Table 4-1, the design of the database and the CIP-25 metadata does not change.

4.4 Trust Assumptions

In the prototype, the opening decision is made by the operator's server rather than by an Aiken validator. Part of the service's central promise, that no one can open a capsule before the specified date and time, therefore rests on trust in the operator rather than on cryptography or the blockchain. This section distinguishes what third parties can verify independently from what relies on trust in the operator or in external services. Figure 4-3 shows the scope of trust in the prototype.

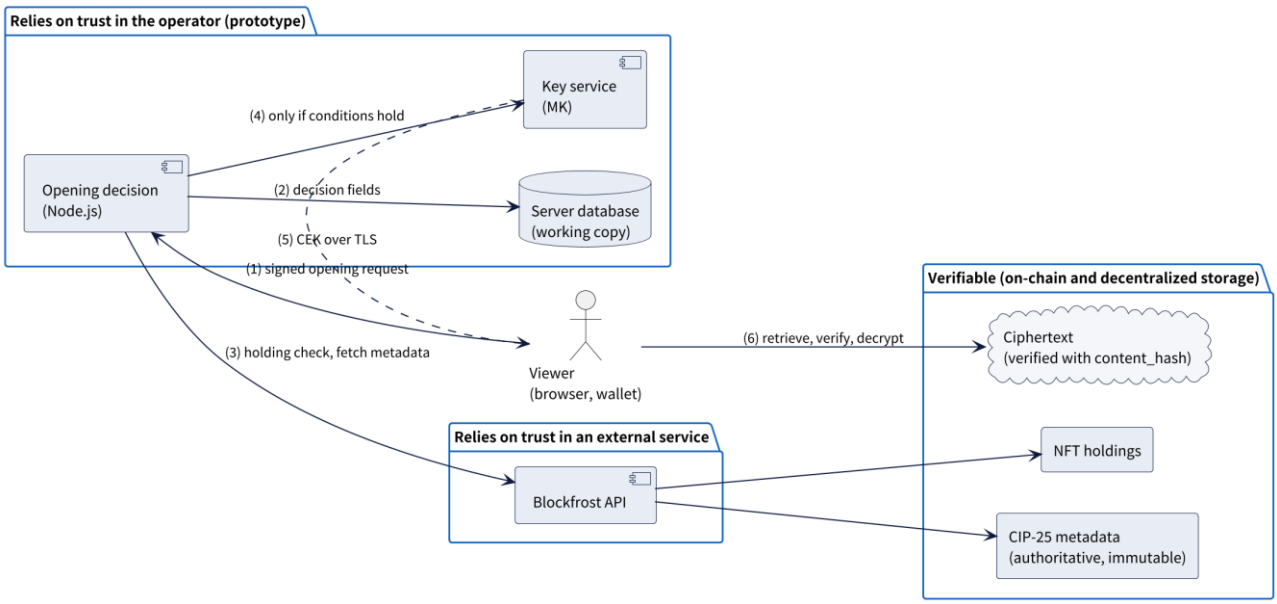


Figure 4-3 Scope of Trust in the Prototype

Item	Prototype	Production (After the Aiken Migration)
The fact and time of sealing	Verifiable. The sealing transaction and sealed_at	Same

Item	Prototype	Production (After the Aiken Migration)
	remain on-chain	
The opening conditions (unlock_at, opening_mode) have not changed since sealing	Verifiable. The CIP-25 metadata cannot be changed, and changes on the database side are detected by the reconciliation in Section 4.3	Same; also recorded in the Datum
The ciphertext has not been tampered with	Verifiable in three stages: content_hash, the GCM authentication tag, and plain_hash	Same
Who may open (the NFT holder)	Relies on the operator. The server decides based on Blockfrost query results	Verifiable. The validator verifies the NFT in the inputs of the opening transaction
The time condition	Relies on the operator. Decided by server time (NTP-synchronized)	Verifiable. The validator verifies the transaction's validity range
No key is released before the opening date/time	Relies on the operator, who manages the key service and the master key	Partly relies on the operator (see below)
Correctness of blockchain data	Relies on Blockfrost	Relies on Blockfrost; mitigated by cross-checking with a second provider (Koios or Maestro) (Output A2, Section 4.3)

Table 4-3 What Is Verifiable and What Relies on Trust

The Aiken migration removes the need to trust the operator for the opening decision (who may open, and when). However, even in production, the content key (CEK) is held by the

key service and released only after the opening transaction is confirmed (Output A2, Chapter 3). Trust that the key service does not release a key before the conditions are met therefore remains in production. This trust is to be reduced step by step through non-exportable keys in a KMS, separation of privileges in the key service, auditing through AccessLog, and escrow distribution using threshold secret sharing (Output A2, Section 3.5).

Based on these trust assumptions, the prototype adopts the following mitigations.

- **Testnet only:**

  The prototype runs only on the Cardano Preprod testnet and does not handle assets of monetary value or content containing sensitive personal information.

- **Disclosure to users:**

  The screen informs users that, at the prototype stage, the opening decision is made by the operator's server (reflected in Output B1).

- **Logging and reconciliation:**

  Every key release decision is recorded in AccessLog, and no key is released if a mismatch between the database and the CIP-25 metadata is detected (Section 4.3).

5. Structure and Example of CIP-25 Metadata

Metadata is recorded under transaction metadata label 721 in the hierarchy policy ID → asset name → attributes. When multiple NFTs are minted in one transaction, the asset names are listed as keys under the same policy ID. The following is an example of one piece certificate and one master certificate for a mode ② group capsule (values are dummies). In the prototype, only piece certificates of individual capsules are issued, and manifest_uri is not set.

```
{
 "721": {
  "<policy_id>": {
   "MB-7K3QX2-P007": {
    "name": "Class 3-A Graduation Capsule #007",
    "image": "ipfs://bafybeigdyrztx... (common image CID)",
    "mediaType": "image/png",
    "description": ["Graduation time capsule of Class 3-A, 2026.", "Opens on March 20, 2036."],
    "version": 1,
    "capsule_id": "clx9p1a2b0000abcd7k3qx2ef",
    "capsule_type": "GROUP",
    "piece_id": "clx9p2c3d0007abcd1m2n3o4p",
    "slot_no": 7,
    "kind": "PIECE",
    "content_type": "IMAGE",
    "content_uri": ["ipfs://bafybeib3k5m7q9r2t4v6x8z0a1c3e5g7i9k1m3o5q7s9u1",
"w3y5a7c9e1g3"],
    "content_hash": "9f2c... (64 digits)",
    "plain_hash": "4a71... (64 digits)",
    "enc": {
     "alg": "AES-256-GCM", "iv": "1a2b3c4d5e6f7a8b9c0d1e2f",
     "tag_len": 128, "key_ref": "k1:clx9p1a2...:clx9p2c3...",
     "kdf": "NONE", "wrap": "AES-KW-256"
    },
```

```
    "manifest_uri": ["ipfs://bafybeih..."],
    "unlock_at": 2089324800000,
    "opening_mode": "MODE_2",
    "sealed_at": 1773878400000,
    "registrant": "addr_tes...x7q2kd",
    "files": [{ "name": "piece-007.enc", "mediaType": "application/octet-stream",
          "src": ["ipfs://bafybeib3k5m7...", "w3y5a7c9e1g3"] }]
   },
   "MB-7K3QX2-M01": {
    "name": "Class 3-A Graduation Capsule Master #01",
    "image": "ipfs://bafybeigdyrztx...", "mediaType": "image/png",
    "version": 1, "capsule_id": "clx9p1a2b0000abcd7k3qx2ef", "capsule_type": "GROUP",
    "kind": "MASTER", "manager_no": 1, "manager_count": 2, "quorum": 0,
    "covers": ["MB-7K3QX2-P001", "MB-7K3QX2-P002", "...", "MB-7K3QX2-P032"],
    "manifest_uri": ["ipfs://bafybeih..."],
    "unlock_at": 2089324800000, "opening_mode": "MODE_2", "sealed_at": 1773878400000
   }
  },
  "version": "1.0"
 }
}
```


Figure 5-1 Example of CIP-25 Metadata (Mode ②, Piece Certificate and Master Certificate)

5.1 Structure of the Manifest JSON (Production)

The manifest is an off-chain document that certifies the whole capsule as a single document. It is referenced from each NFT via manifest_uri, and its hash is held in the database and the Datum. This section is a production specification; the prototype, with one piece per capsule, does not create a manifest (Table 0-1).


```
{
 "version": 1,
 "capsule_id": "clx9p1a2b0000abcd7k3qx2ef",
 "policy_id": "<policy_id>",
```

```
 "capsule_type": "GROUP", "opening_mode": "MODE_2",
 "unlock_at": 2089324800000, "sealed_at": 1773878400000,
 "piece_count": 32, "master_count": 2,
 "pieces": [
  { "asset_name": "MB-7K3QX2-P001", "slot_no": 1, "content_type": "TEXT",
    "content_uri": "ipfs://bafy...", "mirror_uris": ["ar://..."],
    "content_hash": "...", "plain_hash": "...", "size_bytes": 2048 },
  ...
 ],
 "masters": [ { "asset_name": "MB-7K3QX2-M01", "manager_no": 1 }, ... ]
}
```

Figure 5-2 Structure of the Manifest JSON

6. Detailed Design of the Minting Flow

Figure 6-1 shows the flow from the capsule manager's finalize action on SCREEN-08 (NFT Confirmation) until the minting of the NFTs is confirmed. The status display on screen (pending, confirmed, failed) is defined in Output B1.

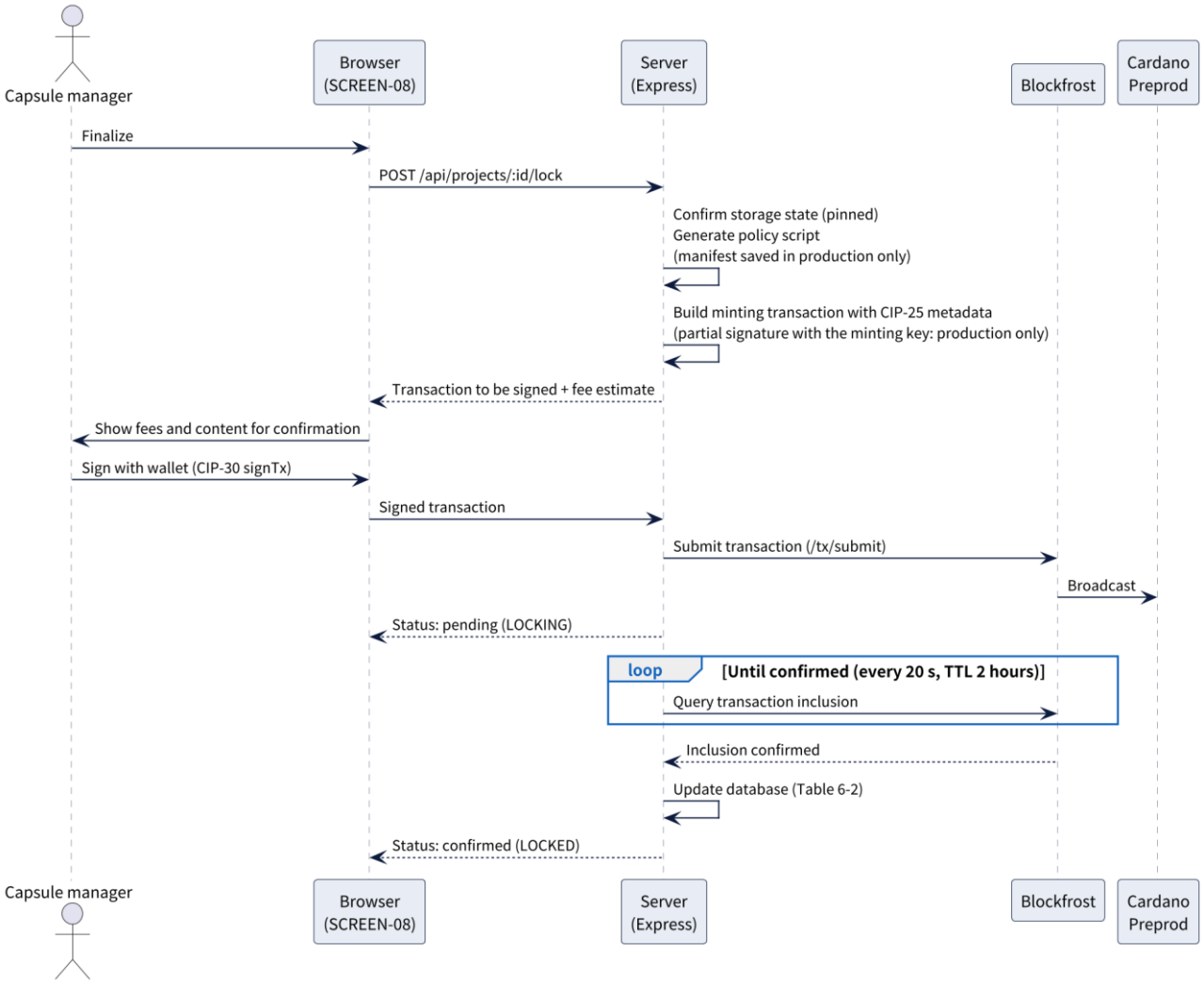


Figure 6-1 Sequence of NFT Minting (Sealing)

6.1 Composition of the Minting Transaction

Element	Content
Inputs	**UTxOs of the capsule manager's wallet (fees plus minimum ADA for each NFT)**
Mint	**Piece certificates × number of pieces, plus master certificates × number of**

Element	Content
	managers (modes ② and ③)
Outputs	One piece certificate plus minimum ADA to each piece registrant's address; one master certificate plus minimum ADA to each manager's address; change returned to the manager.
Metadata	All NFTs recorded under label 721 in the structure of Chapter 5
Signatures	Prototype: the manager's wallet only (covering both payment and the policy condition). Production: the manager's wallet (payment) plus the service-side minting key (policy condition)
Validity (TTL)	Current slot plus 2 hours. If exceeded, the transaction is treated as failed and rebuilt.

Table 6-1 Composition of the Minting Transaction

6.2 Split Minting (Production)

Given Cardano's maximum transaction size (16 KB) and the volume of metadata, the number of NFTs that can be minted safely in one transaction is roughly 20 to 25. For capsules with more pieces, minting is split across multiple transactions under the same policy (e.g., 32 pieces plus 2 managers → 2 transactions). Split minting is handled as follows.

- **Status during split minting:**

  Until all split transactions are confirmed, the capsule is in the LOCKING state (sealing in progress); it transitions to LOCKED when all are confirmed.

- **Partial failure:**

  If some transactions fail, only the failed portion is rebuilt and resent. Portions that have succeeded are not reissued (duplicates are identified by asset name).

- **Timeout:**

  If minting is not completed within the time condition of the policy (default 24 hours), the manager is notified and the case is handled individually by operations.

6.3 State Updates After Minting

Table	Updates
Project	status = LOCKED, policyId, policyScript, sealedAt, mintTxHashes[]
Piece	assetName, mintStatus = CONFIRMED, contentHash, plainHash, manifestUri (production only)
PieceSlot	status = FILLED (empty slots are excluded from minting by default)
Capsule	Created (1:1 with Project): openingMode, quorum, managerCount
MasterToken (new)	One row per master certificate: assetName, managerWallet, managerNo

Table 6-2 Database Updates After Minting Is Confirmed

7. Basis for Estimating Network Fees

This chapter defines the basis for the fee estimate presented to the user before minting on SCREEN-08 (NFT Confirmation) (Output B1, Chapter 4). The amounts are approximations based on the Cardano Preprod protocol parameters; at implementation, they are calculated from the transaction built with the Mesh SDK (actual fee) and the protocol parameters (minimum ADA coefficient).

Cost Item	Calculation Method	Approximate Amount (Preprod, equivalent to Mainnet)
Transaction fee	The fee of the transaction built with the Mesh SDK (proportional to size and execution steps)	About 0.2 to 0.4 ADA per transaction
Minimum ADA (piece certificate)	Minimum ADA required for each output UTxO carrying an NFT; depends on the metadata volume	About 1.2 to 1.5 ADA per NFT (remains as the recipient's asset)
Minimum ADA (master certificate)	Same as above	About 1.2 to 1.5 ADA per NFT
Storage cost (IPFS)	Prototype: Blockfrost IPFS (Starter plan by default; Hobby if needed; Output A2, Section 2.5.3)	No charge to users (operating cost: free on Starter; EUR 29 per month excluding tax on Hobby)
Storage cost (Arweave)	Production: estimated from the byte count using the Turbo pricing API	Proportional to size (Output A2, Chapter 2)

Table 7-1 Fee Items and Calculation Methods

For example, a group capsule with 32 pieces and 2 managers (mode ②) involves 34 NFTs in 2 transactions, for a total of roughly 34 × 1.3 ADA + 2 × 0.3 ADA ≈ 45 ADA (of which about

44 ADA remains in the recipients' wallets as minimum ADA). In the prototype, fees are paid by the capsule manager. For production, the policy is to also offer a plan in which the service pays the fees on the user's behalf (a B2B2C model).

8. Key Design Decisions and Their Rationale

This chapter summarizes the main design decisions made in this document and their rationale.

Decision	Content	Rationale
Encryption method	AES-256-GCM, implemented with the browser-standard Web Crypto API	GCM is authenticated encryption and also detects tampering. The Web Crypto API requires no additional library and keeps key material from being exposed in memory.
Decryption keys	A random content key (CEK) per piece, stored wrapped in the key service	The opening date/time is public information, so a key derived from it could be computed by anyone.
Minting policy	One time-locked native script policy per capsule	The supply is fixed, which increases trust in the certificate, and no Aiken minting validator is needed.
Burning NFTs	Burning is not offered as an operation in the UI	After the time condition of the policy (24 hours) passes, burning is impossible, so a post-burn state does not arise in principle.
Tracking NFT transfers	No continuous tracking; ownership is queried through Blockfrost at the time of the opening request	The decryption condition (Section 3.4) is evaluated on the spot, so continuous tracking is unnecessary.
Empty piece slots (group capsules)	Excluded from minting by default; empty sealing is allowed only when a manager forces sealing	Final confirmation depends on the operational requirements of capsule managers. Not relevant to the prototype, which handles

Decision	Content	Rationale
		individual capsules only.

Table 8-1 Key Design Decisions and Their Rationale

9. Summary

This document defines, as Milestone 2 Output A1, the design table of the information stored as NFTs. First, eight data types were defined in light of the 64-byte constraint of CIP-25 metadata, and the attributes of the piece certificate and the master certificate were specified without placing any personal information on-chain. Next, the links to the stored data, the two-stage hash verification of ciphertext and plaintext, and an enc object that never contains the key itself were defined, followed by the four decryption conditions based on the opening mode and the time conditions expressed in UTC milliseconds. These five items were consolidated into a single design table (Table 3-9). The document also specifies the minting policy, asset naming, minting flow, split minting, and the basis for fee estimates, so that implementation can begin in Milestone 3. In response to the Milestone 1 review, it clarifies where the fields used for the opening decision are held in the prototype, where no on-chain Datum exists, and which parts of the prototype's guarantees rely on trust in the operator, so that what the prototype guarantees can be explained to users and third parties. Finally, the differences between the prototype and production implementations are shown side by side (Table 0-3). The status of each acceptance criterion is shown in Table 9-1.

Acceptance Criterion	How It Is Addressed	Reference
A1-1 Definition of data types	Eight types defined in light of the 64-byte constraint of CIP-25	Section 3.1, Table 3-1
A1-2 Definition of data attributes and content	21 attributes for the piece certificate and 4 for the master certificate. No personal	Section 3.2, Tables 3-2 and 3-

Acceptance Criterion	How It Is Addressed	Reference
	information on-chain	3
A1-3 Links, hashes, and encryption information	Locations via ipfs:// and ar://, two-stage hashes of ciphertext and plaintext, and an enc object that does not contain the key	Section 3.3, Tables 3-4 to 3-6
A1-4 Decryption conditions	Four conditions C-1 to C-4 based on the opening mode, corresponding to the Datum and Redeemer specifications	Section 3.4, Table 3-7, Chapter 4
A1-5 Time conditions	unlock_at, sealed_at, and grace_until in UTC milliseconds, with the staged migration of the decision-maker stated	Section 3.5, Table 3-8
(Reference) Milestone 1 review comment	Where the decision fields are held in the prototype (CIP-25 metadata as authoritative, the database as a working copy, reconciled before key release) and the trust assumptions of the prototype and production	Sections 0.1, 4.3, and 4.4
Consolidated design table	The five items against their locations (metadata, Datum, manifest, database)	Section 3.6, Table 3-9

Table 9-1 Status of Response to the Acceptance Criteria